```python
def my_function():
    """Demonstrates triple quotes docstring and does nothing."""
    return None

print("Using __doc__:")
print(my_function.__doc__)

print("Using help():")
help(my_function)
```

Output

```
Using __doc__:
Demonstrates triple quotes docstring and does nothing.
Using help():
Help on function my_function in module __main__:

my_function()
    Demonstrates triple quotes docstring and does no...
```

Explanation:

- **"""Demonstrates triple quotes docstring and does nothing.""":** Declares the docstring for my_function.
- **my_function.__doc__:** Directly prints the docstring text.
- **help(my_function):** Displays docstring in a structured help format.

Day 5

Iterable vs Iterator:

An iterable is an object that can be looped over, while iterator is the helper object that handles the looping step-by-step.

Iterable	Iterator
Collection/sequence that can return its elements one by one	The specific object that performs the actual traversal
Must implement '***__iter__()***'	Must implement both "***__iter__()***" and "***__next__()***"
Does not keep track of iteration state or current position	Manages internal state and remembers where it is in the sequence
Passed into a "for" loop or "***iter()***" function	Passed into the "***next()***" function to fetch subsequent items.
Eg: lists, tuples, sets, etc	Eg: map, filter, enumerate, etc

- **__iter__()**: returns the iterator object itself.
  - **__next__()**: returns the next value from the sequence.

```
s = "GFG"
it = iter(s)

print(next(it))
print(next(it))
print(next(it))
```

Output

```
G
F
G
```

Steps top create an iterator:

1. Define the class that will act as the iterator.
2. In the "***__init__()***" method, initialise any required attributes that will be used
3. Implementing "***__iter__()***" method should return the iterator object itself
4. Implementing "***__next__()***" method should provide the next item in the sequence each time it's called

List comprehension:

It is a concise way to create new lists by applying an expression to each item in an existing iterable (list, tuple, range). It helps to write readable and efficient code.

```
a = [2, 3, 4, 5]
res = [val ** 2 for val in a]
print(res)
```

Output

```
[4, 9, 16, 25]
```

```
a = [1, 2, 3, 4, 5]
res = [val for val in a if val % 2 == 0]
print(res)
```

Output

```
[2, 4]
```

Dictionary comprehension:

It is used to create a dictionary in a short and clear way. It allows keys and values to be generated from a loop in one line.
This helps in building dictionaries directly without writing multiple statements.

```
sq = {x: x**2 for x in range(1, 6)}
print(sq)
```

Output

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
keys = ['a','b','c','d','e']
values = [1, 2, 3, 4, 5]

d = {k:v for (k,v) in zip(keys, values)}
print (d)
```

Output

```
{'a': 1, 'c': 3, 'b': 2, 'e': 5, 'd': 4}
```

- "**zip()**" function pairs each state with its corresponding element, creating a dictionary.

Set comprehensions:

They are similar to list comprehensions but result in sets, automatically eliminating duplicate values.

```
a = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7]

res = {num for num in a if num % 2 == 0}
print(res)
```

Output

```
{2, 4, 6}
```

```
res = {num**2 for num in range(1, 6)}
print(res)
```

Output

```
{1, 4, 9, 16, 25}
```

Generator expressions:

It is a shorter and more compact way to create a generator without using a function or yield. It looks similar to list comprehension but uses '()'.

Characteristics -

- Produces items one at a time
- Ideal when values need to be iterated only once
- Returns a generator object, not a list
- Faster and more memory-efficient for large data sets

```python
generator = (num ** 2 for num in range(10))
for num in generator:
    print(num)
```

Output

```
0
1
4
9
16
25
36
49
64
81
```

Generator function:

It is a special type of function that returns an iterator object. Generator functions use '***yield***' to produce a series of results.

```
def fun(max):
    cnt = 1
    while cnt <= max:
        yield cnt
        cnt += 1

ctr = fun(5)
for n in ctr:
    print(n)
```

Output

```
1
2
3
4
5
```

Yield:
It is a keyword used to create generator functions. Unlike 'return', yield returns one value at a time and pauses the function. When called again, the function resumes execution from where it was paused.

```
def numbers():
    yield 1
    yield 2
    yield 3

for num in numbers():
    print(num)
```

Output

```
1
2
3
```

```
def fun(m):
    for i in range(m):
        yield i

for n in fun(5):
    print(n)
```

Output

```
0
1
2
3
4
```

map() function:

The '**map()**' function applies a function to every element of an iterable and returns a map object containing the transformed values.

```
def double(n):
    return n * 2

n = [5, 6, 7, 8]
res = map(double, n)
print(list(res))
```

Output

```
[10, 12, 14, 16]
```

filter() function:

The '**filter()**' function selects elements from an iterable for which a given function returns True.

```python
def is_even(n):
    return n % 2 == 0

n = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

en = filter(is_even, n)
print(list(en))
```

Output

```
[2, 4, 6, 8, 10]
```

zip() function:

It is used to combine two or more iterables (lists, tuples, etc) into a single tuple.

Input: *a = ["Liam", "Emma", "Noah"], b = [90, 85, 88]*
Output: *[("Liam", 90), ("Emma", 85), ("Noah", 88)]*

here, elements at the same position from both iterables are grouped together into tuples.

```
a = [1, 2, 3]
b = ['a', 'b', 'c']

# No iterable are passed
res = zip()
print(list(res))

# One iterable is passed
res = zip(a)
print(list(res))

# Two iterables are passed
res = zip(a, b)
print(list(res))
```

Output

```
[]
[(1,), (2,), (3,)]
[(1, 'a'), (2, 'b'), (3, 'c')]
```

```
names = ['Hiro', 'Mila', 'Tariq']
scores = [88, 94]
res = zip(names, scores)
print(list(res))
```

Output

```
[('Hiro', 88), ('Mila', 94)]
```

enumerate() function:

It is used to iterate over an iterable while keeping track of both index and the value. It returns pairs in the form (index, element).

```python
a = ["Python", "Java", "C++"]
for i, v in enumerate(a):
    print(i, v)
```

Output

```
0 Python
1 Java
2 C++
```

```python
a = ["x", "y", "z"]
e = enumerate(a)

print(next(e))
print(next(e))
```

Output

```
(0, 'x')
(1, 'y')
```

sort() and sorted() functions:

- **sorted()** - it takes an iterable such as list, tuple, etc and returns a new list containing its elements in sorted order. The original iterable remains unchanged.

```
numbers = [5, 2, 8, 1, 4]
result = sorted(numbers)
print(result)
```

Output

```
[1, 2, 4, 5, 8]
```

- **sort()** - available for lists and rearranges their elements directly in ascending/descending order. It changes the original list and returns None.

```
numbers = [5, 2, 8, 1, 4]
numbers.sort()
print(numbers)
```

Output

```
[1, 2, 4, 5, 8]
```

sorted()	sort()
Built-in function	List method
Works with any iterable	Works only with lists
Returns a new sorted list	Returns None
Does not modify the original iterable	Modifies the original list
Ascending by default, reverse=True for descending	Ascending by default; reverse=True for descending

any() function:

It return True if any of the elements of a given iterable are True, or else it returns False.

```python
# All elements of list are True
l = [4, 5, 1]
print(any(l))

# All elements of list are False
l = [0, 0, False]
print(any(l))

# Some elements of list are
# True while others are False
# l = [1, 0, 6, 7, False]
# print(any(l))

# Empty list
l = []
print(any(l))
```

Output:

```
True
False
False
```

```python
# All elements of tuple are True
t = (2, 4, 6)
print(any(t))

# All elements of tuple are False
t = (0, False, False)
print(any(t))

# Some elements of tuple are True while
# others are False
t = (5, 0, 3, 1, False)
print(any(t))

# Empty tuple
t = ()
print(any(t))
```

Output:

```
True
False
True
False
```

all() function:

It is a built-in function used to check whether all the elements in the iterable are evaluate to True. It returns True when every element is truthy or when the iterable is empty; otherwise returns False.

```
values = [True, True, False]

result = all(values)

print(result)
```

Output

```
False
```

```
numbers = [10, 20, 30, 40]

result = all(num > 5 for num in numbers)

print(result)
```

Output

```
True
```

Handling large datasets:

It is a common task in data analysis and modification. It is important to use efficient techniques and tools to ensure optimal performance and avoid memory issues.

We can use the following techniques to handle large datasets -

1. Reduce memory usage by optimising data types
2. Splitting data into chunks
3. Use Dask for parallel computing